

```

import torch
import torch.nn as nn
import torchvision.models as models
import torchvision.transforms as transforms
from transformers import BertTokenizer, BertModel
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import random
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import os

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

```

➔ Using device: cpu

```

class MultimodalSentimentDataset(Dataset):
    def __init__(self, tokenizer, transform, max_len=128, size=200):
        self.tokenizer = tokenizer
        self.transform = transform
        self.max_len = max_len
        self.texts = ["I love this!" if i % 2 == 0 else "I hate this!" for i in range(size)]
        self.labels = [1 if i % 2 == 0 else 0 for i in range(size)]
        self.images = [Image.new("RGB", (64, 64), color=("green" if i % 2 == 0 else "red")) for i in range(size)]

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = self.texts[idx]
        image = self.images[idx]
        label = self.labels[idx]

        encoded = self.tokenizer(text, truncation=True, padding="max_length", max_length=self.max_len, return_tensors="pt")
        image = self.transform(image)

        return {
            "input_ids": encoded["input_ids"].squeeze(0),
            "attention_mask": encoded["attention_mask"].squeeze(0),
            "image": image,
            "label": torch.tensor(label, dtype=torch.long),
        }

```

```

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
transform = transforms.Compose([transforms.Resize((64, 64)), transforms.ToTensor()])
dataset = MultimodalSentimentDataset(tokenizer, transform)
train_loader = DataLoader(dataset, batch_size=16, shuffle=True)

```

➔ /usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```

warnings.warn(
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 3.93kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 1.27MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 2.65MB/s]
config.json: 100% 570/570 [00:00<00:00, 47.5kB/s]

```

```

class MultimodalClassifier(nn.Module):
    def __init__(self):
        super(MultimodalClassifier, self).__init__()
        self.text_model = BertModel.from_pretrained("bert-base-uncased")
        self.image_model = models.resnet18(pretrained=True)
        self.image_model.fc = nn.Linear(self.image_model.fc.in_features, 128)

        self.classifier = nn.Sequential(
            nn.Linear(768 + 128, 64),

```

```

        nn.ReLU(),
        nn.Dropout(0.2),
        nn.Linear(64, 2)
    )

    def forward(self, input_ids, attention_mask, images):
        text_outputs = self.text_model(input_ids=input_ids, attention_mask=attention_mask)
        text_feat = text_outputs.pooler_output
        image_feat = self.image_model(images)
        combined = torch.cat((text_feat, image_feat), dim=1)
        return self.classifier(combined)

model = MultimodalClassifier().to(device)

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download
model.safetensors: 100% 440M/440M [00:05<00:00, 74.8MB/s]
/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.11.0. Please use 'weights_only' instead.
warnings.warn(
/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for pretrained are deprecated.
warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth
100%|██████████| 44.7M/44.7M [00:00<00:00, 145MB/s]

optimizer = torch.optim.Adam(model.parameters(), lr=2e-5)
criterion = nn.CrossEntropyLoss()

def train_model(model, loader, epochs=3):
    model.train()
    losses, accs = [], []
    for epoch in range(epochs):
        total_loss, correct = 0, 0
        for batch in loader:
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            images = batch["image"].to(device)
            labels = batch["label"].to(device)

            outputs = model(input_ids, attention_mask, images)
            loss = criterion(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            preds = torch.argmax(outputs, dim=1)
            correct += (preds == labels).sum().item()

        acc = correct / len(loader.dataset)
        print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}, Accuracy: {acc:.4f}")
        losses.append(total_loss)
        accs.append(acc)

    return losses, accs

losses, accs = train_model(model, train_loader)

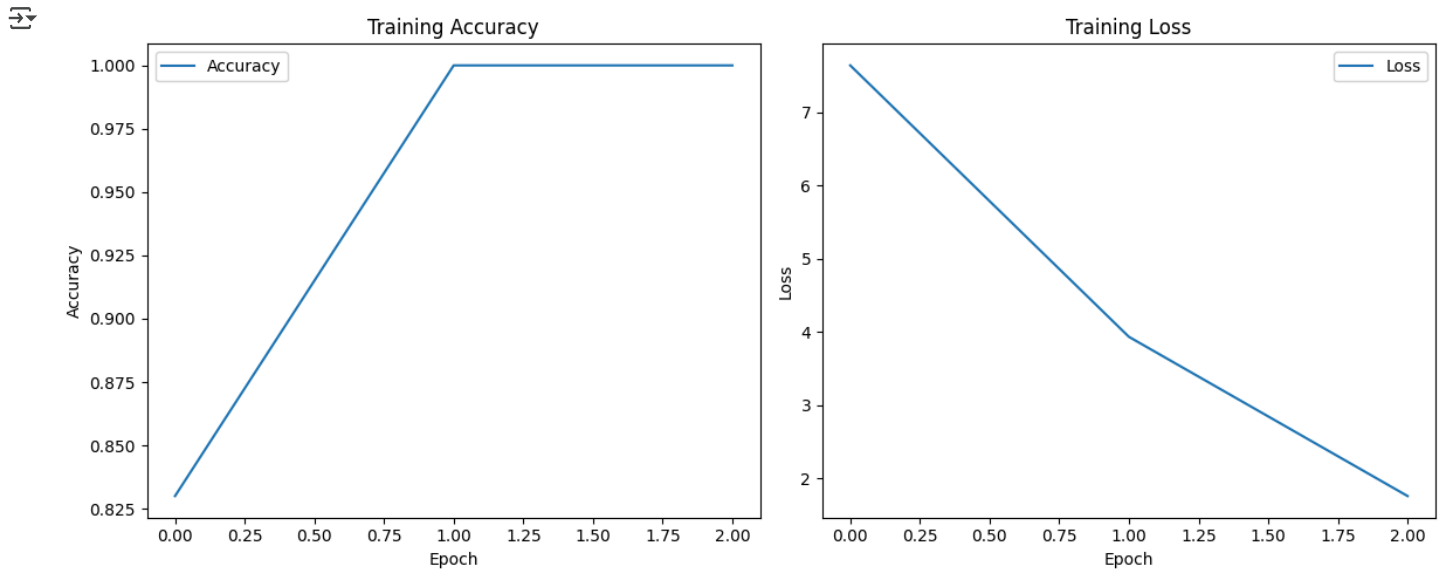
Epoch 1, Loss: 7.6389, Accuracy: 0.8300
Epoch 2, Loss: 3.9336, Accuracy: 1.0000
Epoch 3, Loss: 1.7600, Accuracy: 1.0000

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(accs, label="Accuracy")
plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(losses, label="Loss")
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

```

```
plt.legend()
plt.tight_layout()
plt.show()
```



```
model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for batch in train_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        images = batch["image"].to(device)
        labels = batch["label"].to(device)
        outputs = model(input_ids, attention_mask, images)
        preds = torch.argmax(outputs, dim=1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

print(classification_report(all_labels, all_preds, target_names=["Negative", "Positive"]))
cm = confusion_matrix(all_labels, all_preds)
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues", xticklabels=["Neg", "Pos"], yticklabels=["Neg", "Pos"])
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



	precision	recall	f1-score	support
Negative	1.00	1.00	1.00	100
Positive	1.00	1.00	1.00	100
accuracy			1.00	200
macro avg	1.00	1.00	1.00	200
weighted avg	1.00	1.00	1.00	200

